CONTEXT ENGINEERING FOR ENTERPRISE AI

How to Make AI Understand Your Data

Beyond simple RAG: context engineering, ontologies, GraphRAG and routing for accuracy, cost and scale



Aleksey Savateyev

Cloud & AI Platforms | Microsoft

aka.ms/contextengineering

Microsoft

THE PROBLEM

Why AI agents fail in enterprise workflows

1

Wrong results

Stale indexes and missing data make queries omit current requests, approvals and policy changes; the model may guess

2

No trust or visibility

Users can’t see what data an agent used, what it cost, or why it trusted it

3

Repeated analysis

Without reusable, validated insights, each run repeats work (such as case resolution analysis): more waiting, more cost

The enterprise reality

Truth is scattered across CRM, ERP, HR systems, databases, documents and SharePoint. It changes constantly and sits behind strict access controls, so your AI simply doesn’t understand your data

THE PATH FORWARD

Four disciplines that make AI understand your data

1

Engineer the context

Agent memory and context graphs that know what was true, and when

2

Prime the data

OCR, NER, graph indexing, summarization, clustering

3

Retrieve across data sources

Graph traversal, agentic retrieval, diversity search

4

Route intelligently

Balance accuracy, cost and scale

Shared ontology - one vocabulary for all four: Division · Department · Team · Policy · Task · Employee
Data governance - trusted definitions, named owners and quality gates: Less Rework · Faster Audits · Safer Decisions

Accuracy comes from better context, not just better models.

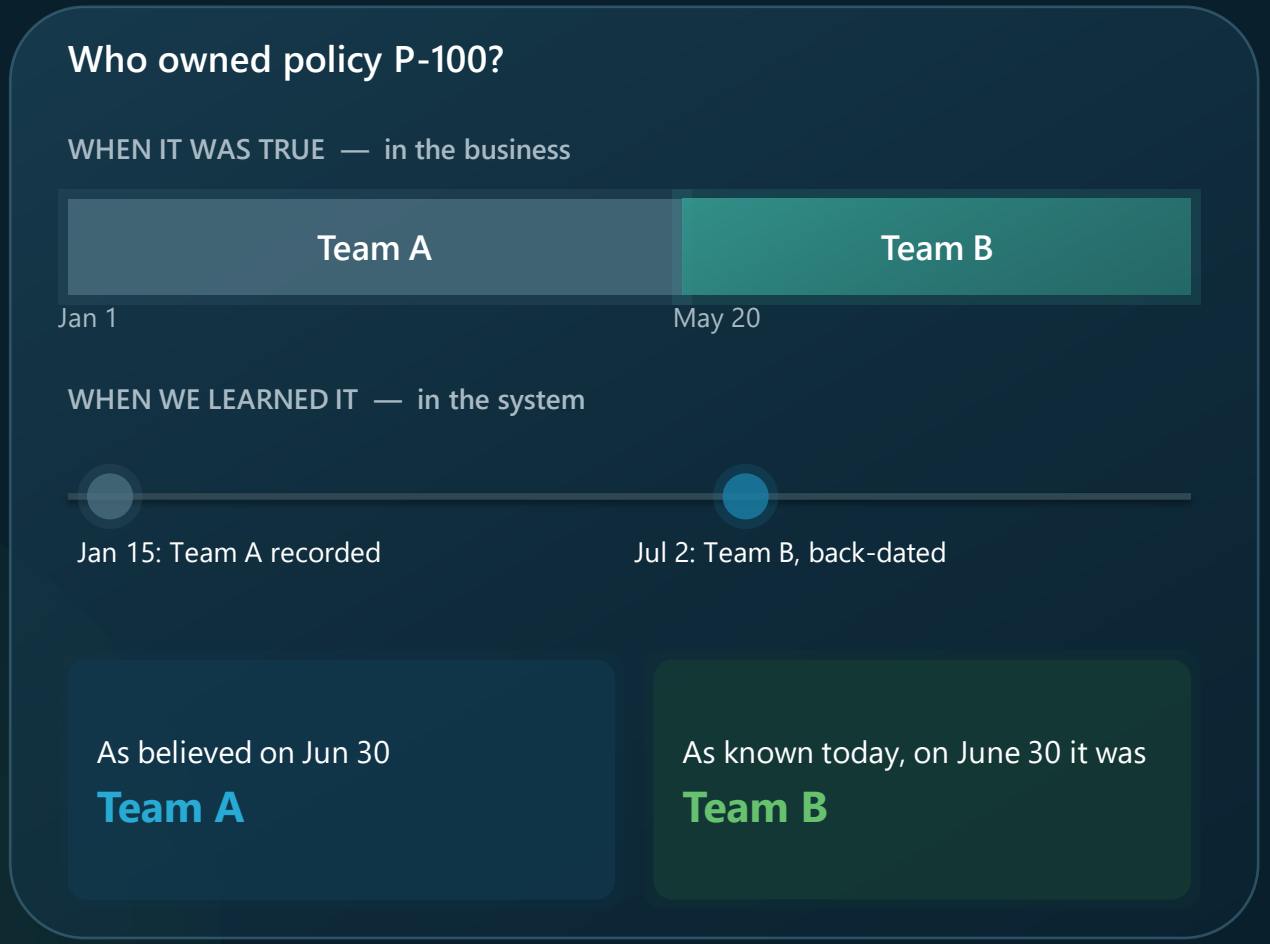
Microsoft

From basic retrieval to context and harness engineering

Basic RAG and Prompt Engineering	Context and Harness Engineering
One search index	Many sources, chosen per question
Stateless prompts	Short- and long-term agent memory
Chunks without relationships	Ontology-typed entities and relationships
Only the “latest” version	What was true, and when we knew it
Stuff the context window	Budgeted context: select, compress, cite

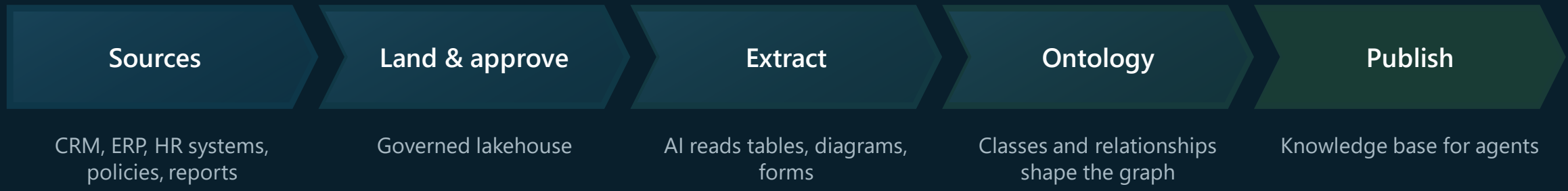
Microsoft

Agent memory on a bi-temporal context graph



- **Separate memory types:** session, past interactions, ontology-typed business facts, reusable skills
- **Track two timelines:** when true and when recorded. Supersede instead of overwriting
- **Record provenance:** source, agent, reviewer
- **Right store for the job:** fast storage for sessions; a governed graph for facts
- **Ask "as of" questions** to reproduce audits and incident reviews

Prime the data before you retrieve it



1 Separate schema from facts

Define superseded-by in the ontology. Store policies P-100 and P-200 as graph nodes, with effective dates and source evidence, so agents can query them.

2 Turn documents into indexable structures

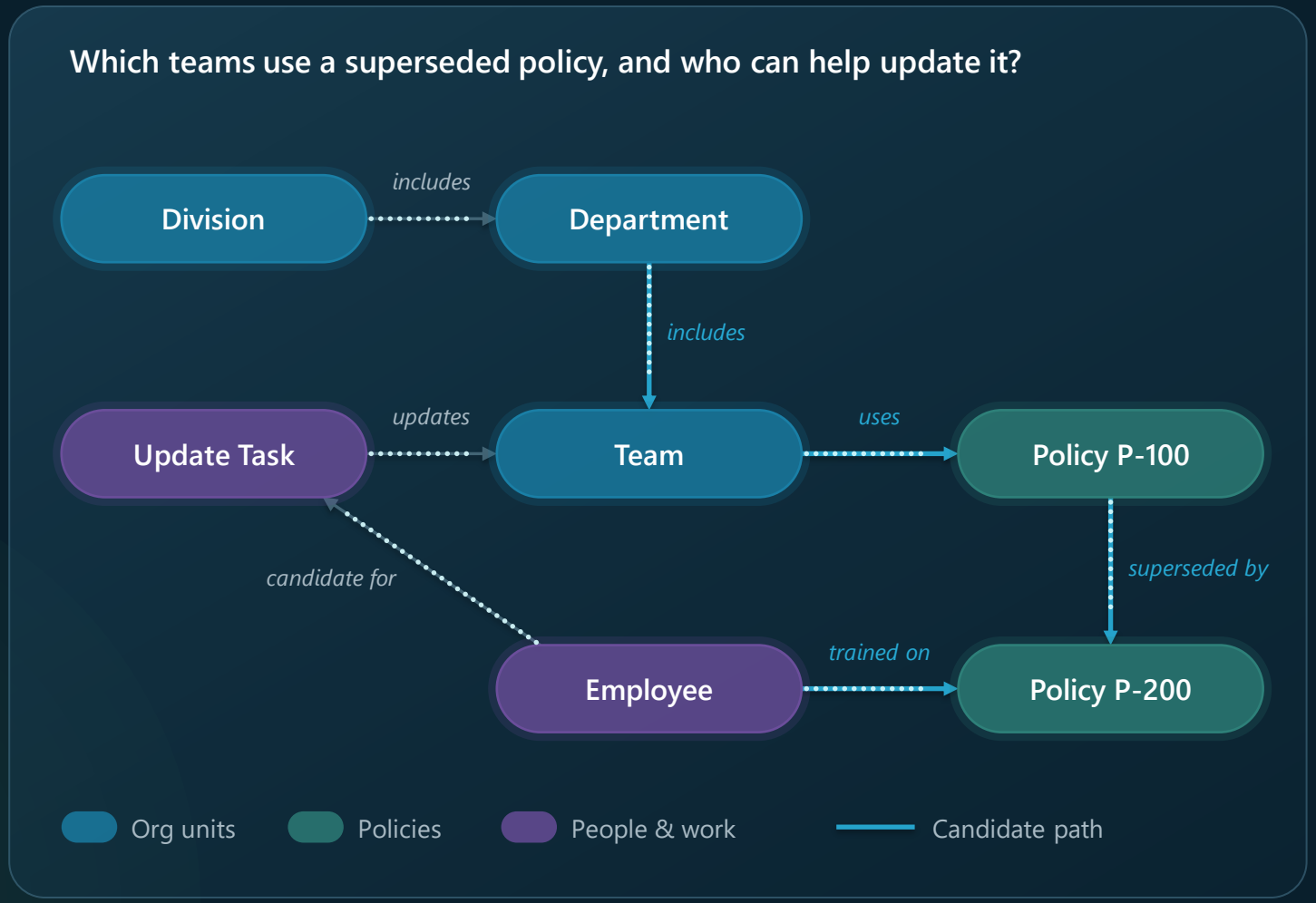
Policies, contracts and forms become table rows or JSON fields (old policy, new policy, effective date) linked to business entities.

3 Let AI tools build, people approve

AI drafts the ontology from policies and business records, merges duplicates (one team, three names) and flags issues. Humans review and sign off.

Microsoft

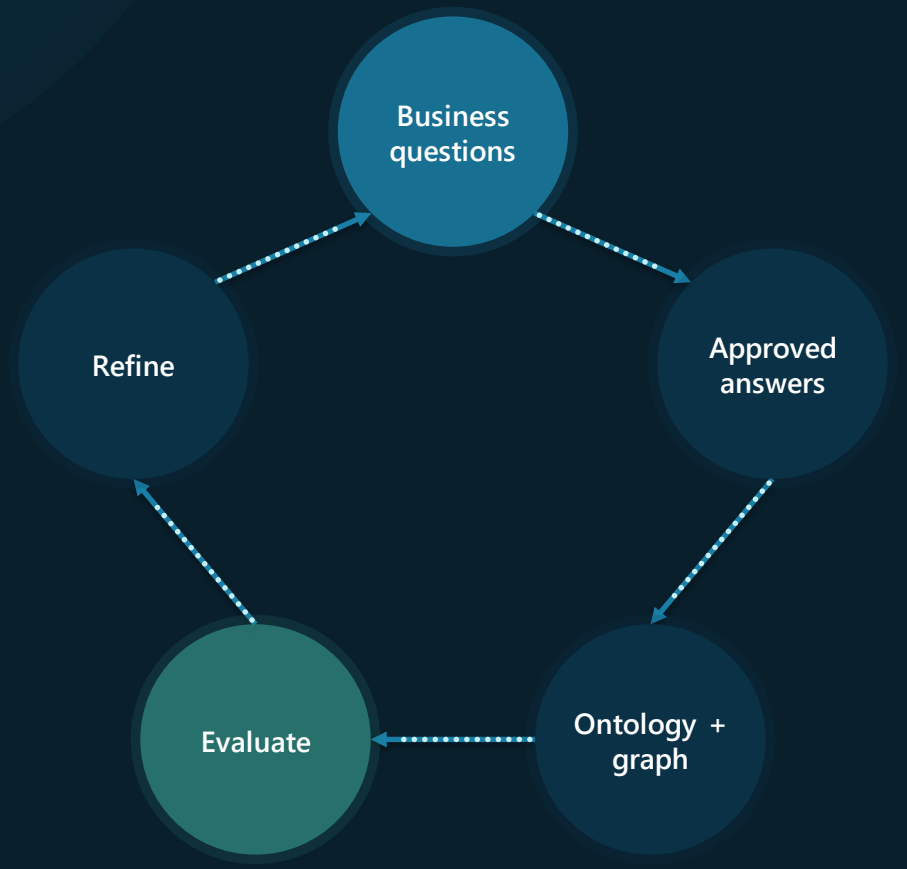
The ontology: one shared language for people and agents



- **Classes:** Division, Department, Team, Policy, Task, Employee
- **Relationships:** includes, uses, superseded by, trained on, candidate for
- **Policy:** enforce approved policy transitions
- **Use shared definitions:** business glossaries and data catalogs
- **Decision gates:** applicability, dates, training, authorization

Microsoft

Let business questions drive the ontology



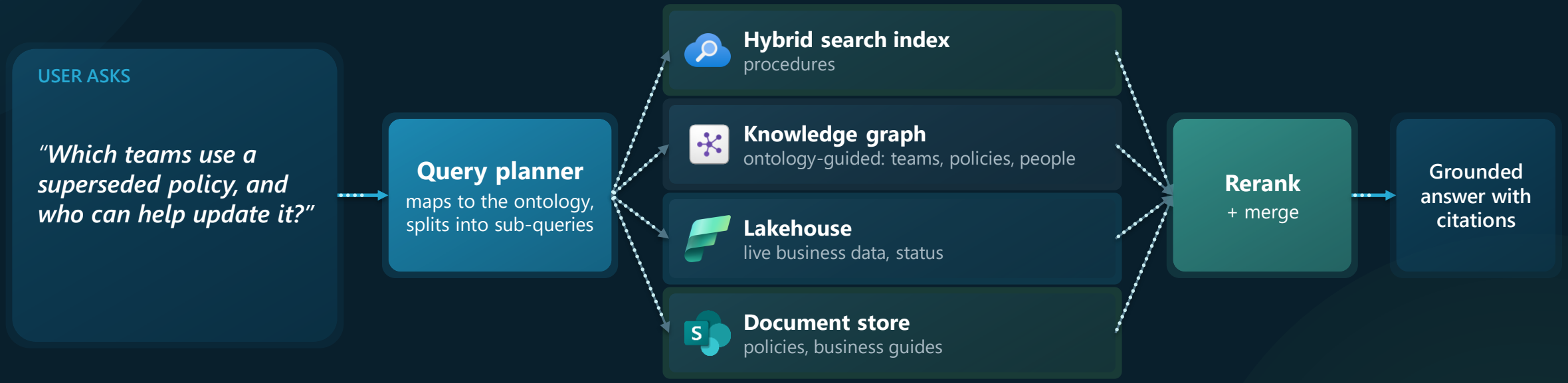
Business-owned test set

Business and compliance leads own questions like “Which teams still use an outdated policy?” and the approved answers.

- **Log tokens and timings** per retrieval step
- **Show the evidence trail:** graph paths and documents behind each claim
- **Capture prompts and retrieved context** for debugging
- **Compare approaches side by side** on the same questions

Microsoft

One question, many silos: agentic hybrid retrieval



Break down multi-step questions

Run sub-questions in parallel across graph, search and live data.

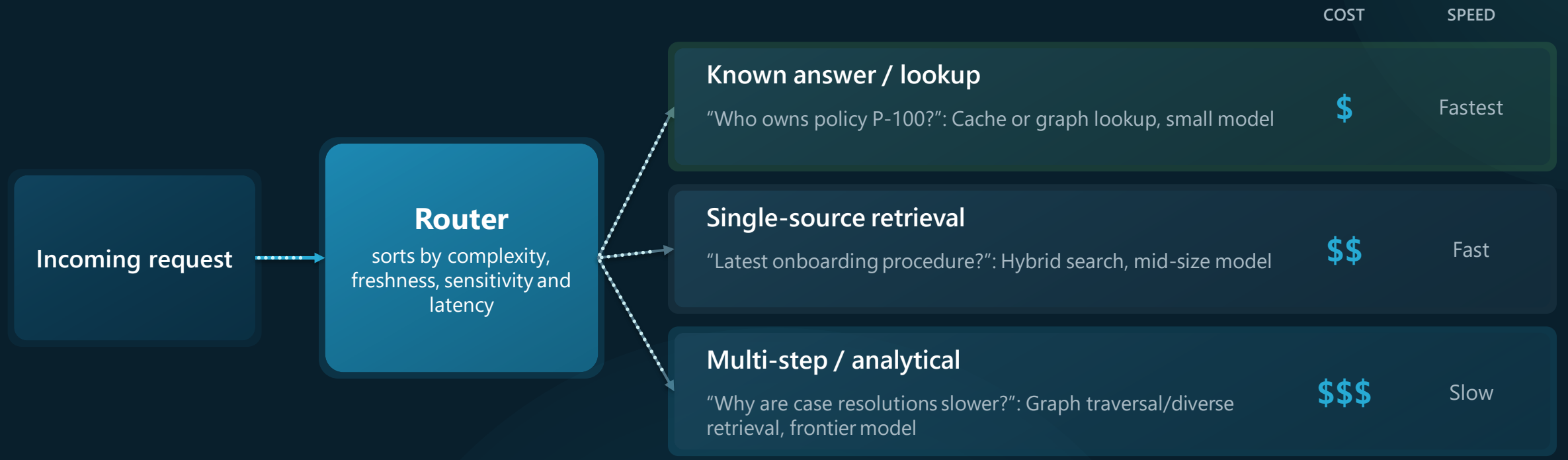
Tune effort per question

Dial reasoning effort; steer each source.

Fresh where it matters

Query live data for status; index what changes slowly.

Route the query, not just the model



Guardrails at the gateway

Caching, token limits and content safety in one place.

Escalate on low confidence

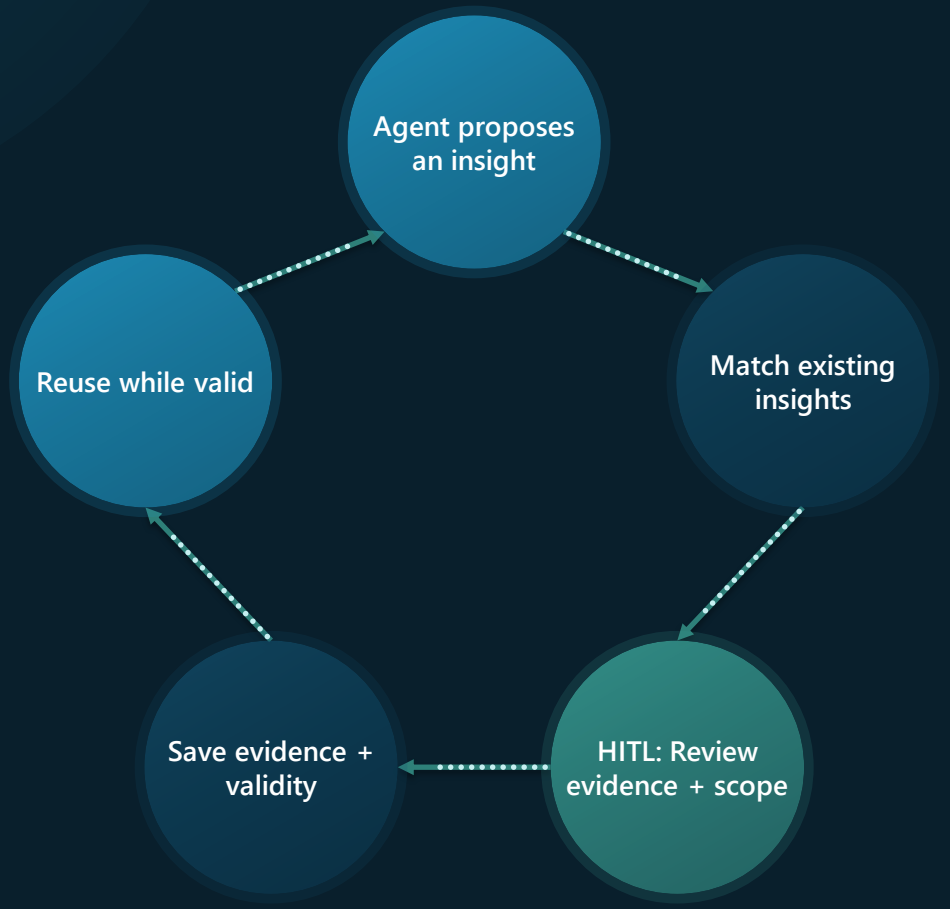
Start cheap; go deeper when evidence is weak.

Evaluate the router

Score routing on the same business questions.

Microsoft

Reuse validated insights with governed write-back



Why it matters

Reviewed observations, such as “Longer approval queues coincide with slower onboarding,” retain evidence, scope and expiry. Reuse can reduce repeat analysis.

Open challenges

- **Discoverability** of insights and skills
- **De-duplication** of overlapping insights
- **Evidence and trust** by source, scope and review status
- **Invalidation** of stale or unsupported insights

PUTTING IT TOGETHER

Enterprise reference architecture*

User experience	Microsoft 365 Copilot	Microsoft Teams	Copilot Studio	
Agents & orchestration	Ontology Agent	Insight Write-Back Agent	Query Routing + Retrieval Microsoft Foundry IQ	
Tools & policy	Graph Indexing Service	Data Pipelines Fabric Data Factory	ReBAC Service Open Policy Agent	
Knowledge & memory	EKG / ontology RDF Graph Database	Unstructured Data Azure AI Search	Structured Data OneLake	Session Store + Insights Azure Cosmos DB

CROSS-CUTTING

- Microsoft Foundry
- Microsoft Fabric
- Microsoft Entra ID
- API Management
- Kubernetes Service
- GitHub Actions

* *Adapt components to your data, governance and deployment requirements*

TAKEAWAYS

Best-practice checklist

1 Context

- ✓ Treat context as a budget: plan, select, compress, cite
- ✓ Track when facts were true and when learned

2 Ontology & priming

- ✓ Let business questions drive the ontology
- ✓ Structure policies, forms and reports before indexing

3 Retrieval

- ✓ Decompose questions across graph, search and live data
- ✓ Enforce identity and permissions at retrieval

4 Routing

- ✓ Route by complexity, freshness, sensitivity and speed
- ✓ Measure cost, speed and evidence. Evaluate continuously

5 Your ontology is a business asset - fund and staff it like one.

- ✓ Talent gap: keep domain owners + an AI adoption lead in-house; partner for graph / AI engineering.

Microsoft

Thank you

Questions?

Context engineering · Ontologies & knowledge graphs

GraphRAG + agentic retrieval · Model and query routing

Aleksey Savateyev

Cloud & AI Platforms | Microsoft



This deck and other Context Engineering best practices

aka.ms/contextengineering